

Modern server grade CPUs like Intel's 4th-generation Sapphire Rapids or above embed AI accelerators like Advanced Matrix Extensions (AMX), which offload matrix multiplications—critical to AI training and inference—liberating CPU cores and delivering substantial performance gains in frameworks such as PyTorch for models like Llama.

RAG unlocks efficiency by querying external, up-to-date sources for real-time or incremental updates, eliminating the need for full retraining. In enterprise settings, models with 3 to 10 billion parameters, fine-tuned to the domain specific knowledge, achieve rapid deployment, lower infrastructure costs, and superior task-specific accuracy without the operational overhead of hyperscale AI.

Enterprises no longer require a single monolithic generative AI model for every function. Instead, they can deploy multiple, domain-specific models, each fine-tuned for a team or department. For instance, a design team could leverage a model optimised for engineering workflows, while HR and finance operate separate models tailored to their processes. Open source platforms such as Llama, Mistral, Falcon, Bloom, or Stable Diffusion can be fine-tuned on industry-specific datasets, delivering higher relevance, faster performance, and lower operational costs.

Model size directly informs hardware selection: lightweight models ($\leq 10\text{B}$ parameters) run efficiently on AI-enabled laptops; medium models ($\sim 20\text{B}$ parameters) are suited for single-socket server CPUs; 100B-parameter models demand multi-socket configurations. Full training of trillion-parameter models remains the preserve of large enterprises.

Software optimisation for existing hardware

Optimising generative AI performance isn't just about

choosing the right model—it's also about knowing how to extract maximum efficiency from the underlying hardware. A simple example illustrates the point.

In a benchmark on an x86 architecture server CPU with 40 cores, parsing a 2.4GB CSV file using Pandas (an open source Python library used for working with datasets) took around 340 seconds. With just a single line change in existing code to 'import modin.pandas' instead of 'pandas' in order to use the optimised version of Pandas called modin, the parse time dropped to 31 seconds, delivering a 10x speed-up without altering the rest of the code. This small optimisation leveraged hardware parallelism more effectively, cutting processing time dramatically.

The takeaway for AI engineers is not to focus solely on the model or framework. Investigate what hardware the client is using, study its acceleration features, and apply relevant optimisations, whether through libraries such as modin, IPEX, hardware-aware compilers, or vendor-specific AI toolkits.

Open ecosystem standards: OPEAI and beyond

Exploring generative AI often faces a steep hardware optimisation curve—especially when tuning models for specific industry needs. One way to simplify the process is through

the Open Platform for Enterprise AI (OPEAI). Backed by the Linux Foundation, OPEAI is a collaboration between major hardware and software players such as Intel, AMD, SAP, Infosys, Wipro, Docker, and others.

The initiative delivers open source architecture blueprints, reference implementations, and benchmarking tools for standard genAI models. More than 30 enterprise use cases—from banking chatbots to manufacturing defect detection—are already covered. Crucially, these resources come pre-optimised for a range of hardware platforms, meaning that enterprises can avoid the trial-and-error of manual tuning.

By leveraging OPEAI's open source repository at opeai.dev, teams can reduce both development time and infrastructure costs while maintaining performance. The approach complements techniques like fine-tuning (for injecting large volumes of domain-specific historical data) and RAG (for feeding real-time external data into the model). Together, these methods enable smaller, cost-efficient AI models to achieve enterprise-grade accuracy without the operational burden of scaling up to trillion-parameter behemoths.

In conclusion, optimising enterprise genAI means structuring workloads so that each task is handled by a model tailored to its domain. By aligning model design with specific operational goals and integrating RAG for real-time relevance, organisations create leaner, faster, and more accurate systems, maximising both hardware efficiency and the value of their data without overextending resources. **END** 🐼

*This article is based on the talk titled 'Optimising Generative AI for Hardware' delivered by **Anish Kumar**, AI software engineering manager, Intel Technology India Pvt Ltd, at AI DevCon 25 in Bengaluru. It has been transcribed and curated by **Janarthana Krishna Venkatesan**, journalist at OSFY.*